

BCA 5th Semester

PHP

Topic : PHP Basics

(Part-2)

- By Soudip Sir

2

PHP Basics: PHP syntax, variables, data types, operators, and control structures. Functions in PHP: Defining and using functions, function scope, including files, and built-in functions
Arrays and Strings: Working with arrays, array functions, string manipulation, and string functions

Returning Values in PHP Functions

Functions in PHP can return a value using the return statement. The return value can be any data type such as a string, integer, array, or object. If no return statement is provided, the function returns null by default.

- The function multiply() is defined to return the product of \$a and \$b.
- The function is called with 4 and 5 as arguments.
- The result 20 is returned and printed using echo.

```
<?php
    function multiply($a, $b) {
        return $a * $b;
    }
    $result = multiply(4, 5); // $result will be 20
    echo $result; // Output: 20
?>
```

Output

20

Anonymous Functions (Closures)

PHP supports anonymous functions, also known as closures. These functions do not have a name and are often used for passing functions as arguments to other functions.

- An anonymous function is defined and assigned to the variable \$greet.
- The function takes \$name as a parameter and prints a greeting message.
- The function is called with the argument "GFG",
and the greeting "Hello, GFG" is printed.

```
<?php
✓ $greet = function($name) {
    echo "Hello, " . $name . "!";
};
$greet("GFG");
?>
```

Output

Hello, GFG!

Recursion in PHP

Recursion is a process in which a function calls itself. It is often used in situations where a task can be broken down into smaller, similar tasks.

- The function `factorial()` is defined to calculate the factorial of a number `$n` using recursion.
- If `$n` is 0, the function returns 1 (base case); otherwise, it calls itself with `$n - 1`.
- When `factorial(5)` is called, the function recursively multiplies $5 * 4 * 3 * 2 * 1$, returning 120.



```
<?php
function factorial($n) {
    if ($n == 0) {
        return 1;
    } else {
        return $n * factorial($n - 1);
    }
}
echo factorial(5);
?>
```

Output

120

Common PHP Functions

Here are some common PHP functions you should know:

1. String Functions

- **strlen()**: Returns the length of a string.
- **strtoupper()**: Converts a string to uppercase.
- **strtolower()**: Converts a string to lowercase.
- **substr()**: Returns a part of a string.

2. Array Functions

- **array_push()**: Adds elements to the end of an array.
- **array_pop()**: Removes the last element of an array.
- **array_merge()**: Merges two or more arrays.

3. Date and Time Functions

- **date()**: Returns the current date or time.
 - **strtotime()**: Converts a string into a Unix timestamp.
 - **time()**: Returns the current Unix timestamp.
- 

INCLUDING FILES

When applications become large, we don't want everything inside one PHP file.
We can divide code into multiple files.

For example:

project

├── index.php

├── header.php

├── footer.php

└── database.php

PHP provides:

include ✓

require ✓

include_once

require_once

include

```
include "header.php";
```

PHP includes the contents of that file.

If the file cannot be found, include produces a warning, and execution can generally continue.

require

```
require "database.php";
```

If the required file cannot be loaded, PHP generates a fatal error and execution stops.

include	require
Missing file → warning ✓	Missing file → fatal error ✓
Execution generally continues ✓	Execution stops ✓
Useful for optional components ✓	Useful for essential components ✓

include_once and require_once

These prevent the same file from being included multiple times.

```
include_once "header.php";
```

or:

```
require_once "database.php";
```

require_once is commonly useful for database/configuration files.

PHP Arrays

PHP Arrays

Arrays are one of the most important data structures in PHP. They allow you to store multiple values in a single variable. PHP arrays can hold values of different types, such as strings, numbers, or even other arrays. Understanding how to use arrays in PHP is important for working with data efficiently.

- ✓ • PHP offers many built-in array functions for sorting, merging, searching, and more.
- ✓ • PHP Arrays can store values of different types (e.g., strings, integers, objects, or even other arrays) in the same array.
- ✓ • They are dynamically sized.
- ✓ • They allow you to store multiple values in a single variable, making it easier to manage related data.

Types of Arrays in PHP

There are three main types of arrays in PHP:

1. Indexed Arrays

Indexed Arrays use numeric indexes starting from 0. These arrays are ideal when you need to store a list of items where the order matters.

Output

apple

```
<?php
    $fruits = array("apple", "banana", "cherry");
    echo $fruits[0]; // Outputs: apple
?>
```

2. Associative Arrays

Associative Arrays use named keys, which are useful when you want to store data with meaningful identifiers instead of numeric indexes.

Output

GFG

```
<?php
    $person = array("name" => "GFG", "age" => 30, "city" => "New York");
    echo $person["name"];
?>
```

3. Multidimensional Arrays

Multidimensional Arrays are arrays that contain other arrays as elements. These are used to represent more complex data structures, such as matrices or tables.

Output

22

```
<?php
    $students = array(
        "Anjali" => array("age" => 25, "grade" => "A"),
        "GFG" => array("age" => 22, "grade" => "B")
    );
    echo $students["GFG"]["age"]; // 22
?>
```

Creating Array in PHP

In PHP, arrays can be created using two main methods:

✓ 1. Using the `array()` function

The traditional way of creating an array is using the `array()` function.

```
$fruits = array("apple", "banana", "cherry");
```

✓ 2. Using short array syntax (`[]`)

In PHP 5.4 and later, you can use the shorthand `[]` syntax to create arrays.

```
$fruits = ["apple", "banana", "cherry"];
```

You can also create associative arrays by specifying custom keys:

```
$person = ["name" => "GFG", "age" => 30]; ✓
```

Accessing and Modifying Array Elements

✓ 1. Accessing Array Elements

You can access individual elements in an array using their index (for indexed arrays) or key (for associative arrays).

- **Accessing Indexed Array:**

```
$fruits = ["apple", "banana", "cherry"];  
echo $fruits[0]; // Outputs: apple
```

- **Accessing Associative Array:**

```
$person = ["name" => "GFG", "age" => 30];  
echo $person["name"]; // Outputs: GFG
```


2. Modifying Array Elements

You can modify an existing element by assigning a new value to a specific index or key.

- **Modifying Indexed Array Element:**

```
$fruits = ["Apple", "Banana", "Cherry"];  
$fruits[1] = "Mango"; // Changes "Banana" to "Mango"  
echo $fruits[1]; // Outputs: Mango
```

- **Modifying Associative Array Element:**

```
$person = ["name" => "GFG", "age" => 25];  
$person["age"] = 26; // Updates the age to 26  
echo $person["age"]; // Outputs: 26
```

Adding and Removing Array Items

1. Adding Array Elements

You can add new elements to an array using the following methods:

- **array_push():** Adds elements to the end of an indexed array.

```
$fruits = ["apple", "banana"];  
array_push($fruits, "cherry"); // Adds "cherry" to the end
```

- **array_unshift():** Adds elements to the beginning of an indexed array.

```
array_unshift($fruits, "pear"); // Adds "pear" to the beginning
```

- **Direct assignment:** Adds an element to an associative array.

```
$person["city"] = "New York";
```

✓ 2. Removing Array Elements

To remove items from an array, you can use several functions:

- **array_pop()**: Removes the last element from an indexed array.

```
array_pop($fruits);
```

- **array_shift()**: Removes the first element from an indexed array.

```
array_shift($fruits);
```

- **unset()**: Removes a specific element from an array by key or index.

```
unset($fruits[2]); // Removes the element with index 2
```

Array Functions

PHP provides a wide range of built-in functions to work with arrays. Here are some common array functions:

- ✓ **Array Merge:** The `array_merge()` function combines two or more arrays into one.

```
$array1 = [1, 2, 3];
```

```
$array2 = [4, 5, 6];
```

```
$merged = array_merge($array1, $array2);
```

```
print_r($merged); // Outputs: [1, 2, 3, 4, 5, 6]
```

- **Array Search:** The `in_array()` function checks if a specific value exists in an array.

```
$fruits = ["Apple", "Banana", "Cherry"];  
if (in_array("Banana", $fruits)) {  
    echo "Banana is in the array!";  
}
```

- **Array Sort:** The `sort()` function sorts an indexed array in ascending order.

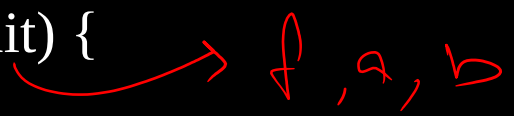
```
$numbers = [3, 1, 4, 1, 5];  
sort($numbers);  
print_r($numbers); // Outputs: [1, 1, 3, 4, 5]
```

Array Iteration

You can loop through arrays using loops such as foreach or for.

✓ **Using foreach Loop:**

```
$fruits = ["Apple", "Banana", "Cherry"]; ✓  
foreach ($fruits as $fruit) {  
    echo $fruit . "<br>";  
}
```



✓ **Using for Loop:**

```
$numbers = [1, 2, 3, 4];  
for ($i = 0; $i < count($numbers); $i++) {  
    echo $numbers[$i] . "<br>";  
}
```

PHP - Array Sorting Functions

The items in an array can be sorted in alphabetical or numerical order, descending or ascending.

Here are the main PHP array sorting functions:

- ✓ • `sort()` - sorts an indexed array in ascending order
- ✓ • `rsort()` - sorts an indexed array in descending order
- ✓ • `asort()` - sorts an associative array in ascending order, according to the value
- ✓ • `ksort()` - sorts an associative array in ascending order, according to the key
- ✓ • `arsort()` - sorts an associative array in descending order, according to the value
- ✓ • `krsort()` - sorts an associative array in descending order, according to the key

1. `sort()` — Ascending Order

Sorts an indexed array in ascending order.

Example

```
<?php
```

```
$numbers = [50, 20, 40, 10, 30];
```

```
sort($numbers);
```

```
print_r($numbers);
```

```
?>
```

Output

Array

(

[0] => 10

[1] => 20

[2] => 30

[3] => 40

[4] => 50

)

2. **rsort()** — Descending Order

Sorts an **indexed array** in descending order.

```
<?php
```

```
$numbers = [50, 20, 40, 10, 30];
```

```
rsort($numbers);
```

```
print_r($numbers);
```

```
?>
```

Output

Array

(

[0] => 50

[1] => 40

[2] => 30

[3] => 20

[4] => 10

)

Associative Array Sorting

Consider this associative array:

```
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];
```

Here:

Key	Value
-----	-------

Rahul → 85

Priya → 92

Amit → 78

Sneha → 88

Now we can sort either by **value** or by **key**.

3. asort() — Value Ascending

Sorts an associative array according to its **values in ascending order**.

```
<?php
```

```
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];  
asort($students);  
print_r($students);  
?>
```

Output

```
Array  
(  
    [Amit] => 78  
    [Rahul] => 85  
    [Sneha] => 88  
    [Priya] => 92  
)
```

4. arsort() — Value Descending

Sorts an associative array according to its **values in descending order**.

```
<?php
```

```
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];  
  
arsort($students);  
print_r($students);  
?>
```

Output

```
Array  
(  
    [Priya] => 92  
    [Sneha] => 88  
    [Rahul] => 85  
    [Amit] => 78  
)
```

Again, the keys are preserved.

5. ksort() — Key Ascending

Sorts an associative array according to its **keys in ascending order**.

```
<?php
```

```
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];  
  
ksort($students);  
print_r($students);  
?>
```

Output

```
Array  
(  
    [Amit] => 78  
    [Priya] => 92  
    [Rahul] => 85  
    [Sneha] => 88  
)
```



Notice that we are sorting the **names**, not the marks.

6. krsort() — Key Descending

Sorts an associative array according to its **keys in descending order**.

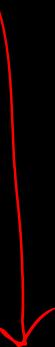
```
<?php
```

```
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];  
krsort($students);  
print_r($students);  
?>
```

Output

Array

```
(  
    [Sneha] => 88  
    [Rahul] => 85  
    [Priya] => 92  
    [Amit] => 78  
)
```



STRING MANIPULATION

What is a String?

A string is a sequence of characters. ✓

\$name = "Great Heights"; ✓

Strings are extremely important in PHP because web applications process a lot of textual data:

- Names
- Emails
- Addresses
- Passwords
- Messages
- URLs
- Search terms

String Concatenation

Concatenation means **joining strings together**.

PHP uses:

.

for concatenation.

Example:

```
$firstName = "Rahul";
```

```
$lastName = "Das";
```

```
echo $firstName . " " . $lastName;
```

Output:

Rahul Das

String Interpolation

Double-quoted strings can directly contain variables.

```
$name = "Rahul";
```

```
echo "Hello $name";
```

Output:

Hello Rahul

With curly braces, you can make variable boundaries clearer:

```
echo "Hello {$name}";
```

String Functions

Function	Purpose	Example	Result
✓ <code>strlen()</code>	Returns the length of a string	<code>strlen("Hello")</code>	5 ✓
✓ <code>str_word_count()</code>	Counts the number of words	<code>str_word_count("Hello World")</code>	2 ✓
✓ <code>strtoupper()</code>	Converts string to uppercase	<code>strtoupper("hello")</code>	HELLO ✓
✓ <code>strtolower()</code>	Converts string to lowercase	<code>strtolower("HELLO")</code>	hello ✓

✓ ucfirst()	Converts the first character to uppercase	ucfirst("hello")	Hello
✓ ucwords()	Converts the first character of each word to uppercase	ucwords("hello world")	Hello World ✓
✓ trim()	Removes whitespace from both ends	trim(" Hello ")	Hello ✓
✓ ltrim()	Removes whitespace from the left side	ltrim(" Hello")	Hello ✓
✓ rtrim()	Removes whitespace from the right side	rtrim("Hello ")	Hello ✓

✓ str_replace()	Replaces one string with another	str_replace("PHP", "Java", "I love PHP")	I love Java ✓
✓ strpos()	Finds the position of the first occurrence of text	strpos("Hello World", "World")	6 ✓
✓ strrpos()	Finds the position of the last occurrence	strrpos("Hello Hello", "Hello")	6
✓ substr()	Extracts a portion of a string	substr("Hello World", 0, 5)	Hello ✓
✓ str_contains()	Checks whether a string contains specific text	str_contains("Hello World", "World")	true ✓
✓ explode()	Splits a string into an array	explode(";", "PHP,MySQL,HTML")	["PHP", "MySQL", "HTML"]
✓ implode()	Joins array elements into a string	implode(" ", ["PHP", "MySQL"])	PHP, MySQL
✓ str_split()	Splits a string into individual characters	str_split("Hello")	["H", "e", "l", "l", "o"]

strcmp()	Compares two strings	strcmp("Hello","Hello")	0 ✓
strcasecmp()	Compares two strings without considering case	strcasecmp("Hello","hello")	0 ✓
str_repeat()	Repeats a string a specified number of times	str_repeat("Hi ",3)	Hi Hi Hi
str_pad()	Adds characters to make a string a specified length	str_pad("PHP",6,"*")	PHP***
htmlspecialchars()	Converts special HTML characters into safe HTML entities	htmlspecialchars(" Hi ")	<h1>Hi</h1>